



Data Structures and Algorithms in Python

Lecture 1: Introduction

Faculty of Data Science and Artificial Intelligence

College of Technology - National Economics University



Some Facts

AI can help write code, but ...

- ① understanding why and how fast it runs is still on us.
- ② it can't replace clear reasoning, pattern recognition, and decision-making.

Your value lies not in writing code, but in thinking critically-analyzing problems, designing solutions, and guiding AI to produce the right code.

... and that's why you should learn

Data Structures and Algorithms



Today: the big picture first

- ① What is Data Structure?
- ② What is Algorithm?
- ③ Why are they closely related? Why are they important?



A short definition

Data structure

A way to **organize and store data** so it can be accessed and modified effectively.

Algorithm

A finite, unambiguous sequence of **steps for solving a problem**.

Data Structures + **Algorithms** = Program



Algorithms use data structures to speed up operations

A common scenario

Given a collection of records, determine whether an element x is present. If it is present, retrieve the information associated with it.

Real-world examples

- Find a student using a student ID.
- Find a customer using a phone number.
- Check whether a username is already registered.
- Find a product and retrieve its price.

A basic approach

Examine the records one by one:

- 1 Compare the current record with x .
- 2 If they match, return the associated information. Otherwise, continue with the next record.
- 3 Report that x is not found after reaching the end.

What if the query is repeated?

But what happens when the collection is large and the same type of search must be performed thousands or millions of times?



Algorithms use data structures to speed up operations

A suitable data structure organizes the records in advance so that repeated search and retrieval operations can be performed more efficiently.



Algorithms use data structures to speed up operations

Example: searching among one million records

Suppose the collection contains $n = 1,000,000$ records and the requested key is not present.

Organization	Approximate work	Explanation
Unsorted sequence	Up to 1,000,000 comparisons	The algorithm may need to examine every record before concluding that the key is absent.
Balanced binary search tree	About 20 comparisons	Each comparison eliminates approximately half of the remaining search space: $\log_2(1,000,000) \approx 20$.
Hash table	Often only a few probes on average	In a well-designed table, a search may require only about 2–3 probes on average.



Example of Data Structure 1

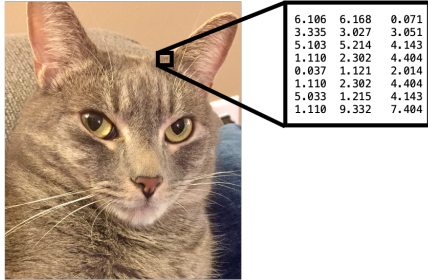


Figure 1: Image representation as a matrix

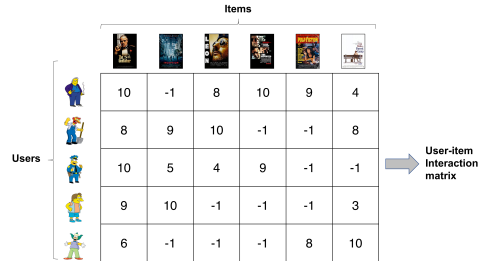
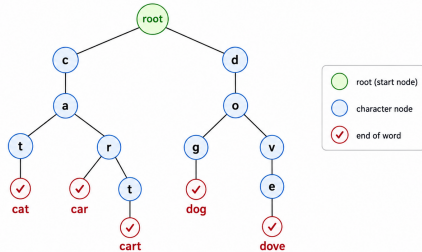


Figure 2: User-rating preferences matrix

Example of Data Structure 2

Trie (Prefix Tree)

Stores words by sharing common prefixes



- Each edge adds one character.
- A path from root to an end node forms a word.
- Words with the same prefix share part of the path.

Figure 3: Words stored in a tree-based data structure

Disjoint Set Union (DSU)

Keeps elements in separate groups and supports find and union

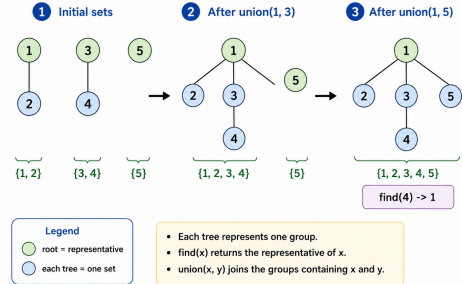


Figure 4: Objects stored in an array-based but tree-like data structure



Common operations and suitable data structures

Common operation	Typical question	Suitable data structures
Store and access by position	What is the value at position i ?	Array, list, matrix
Search and membership	Does x exist? What information is associated with key x ?	Hash table, set, binary search tree
Insert and remove elements	Which element should be added or removed next?	Linked list, stack, queue
Select or prioritize elements	Which element is largest, smallest, or most urgent?	Heap, priority queue, binary search tree
Represent hierarchical relationships	What is the parent, child, or descendant of an object?	Tree, binary tree
Represent general relationships	Are two objects connected? Is there a path between them?	Graph



Advanced operations and suitable data structures

Common operation	Typical question	Suitable data structures / techniques
Merge and compare groups	Are x and y in the same group? How can two groups be joined?	Disjoint Set Union (DSU)
Find common ancestors	What is the closest common ancestor of nodes u and v ?	LCA, binary lifting, Euler tour
Query and update a range	What is the sum, minimum, or maximum from position l to r ?	Segment tree, Fenwick tree
Calculate prefix information	What is the total from the beginning to position i ?	Fenwick tree, prefix-sum array
Search by prefix	Which words start with a given group of letters?	Trie
Answer many queries on fixed data	What is the minimum or maximum value between positions l and r ?	Sparse table

Key idea



Do we need to implement every data structure ourselves?

Good news

Most programming languages already provide efficient implementations of many fundamental data structures.

In Python, for example:

- `list`: dynamic array and basic stack
- `dict`: hash table for key-value data
- `set`: hash-based collection of unique values
- `collections.deque`: queue and double-ended queue
- `heapq`: priority queue implemented using a heap

We can use these data structures without implementing all their internal details from scratch.



Using a data structure still requires understanding

Bad news

Having data structures available does not automatically tell us:

- which data structure is suitable for a problem;
- which operations are efficient or inefficient;
- how much time and memory each operation requires;
- how the structure behaves in special cases.

Example

Both a Python `list` and a `set` can store a collection of values. However:

- searching a list may require examining many elements;
- searching a set is usually much faster on average;
- a list preserves order, while a set does not.



Using a data structure still requires understanding

Choosing the wrong data structure can make an otherwise correct program inefficient.



Why do we study data structures?

Use existing structures

We need to understand how to:

- select a suitable structure;
- use its operations correctly;
- analyze its time and memory costs;
- combine multiple structures.

Develop new solutions

In some applications, existing structures may not directly support the required operations.

We may need to:

- adapt an existing structure;
- extend it with new operations;
- combine several structures;
- design a specialized data structure.

We study data structures not only to implement them, but also to select, use, analyze, adapt, and extend them appropriately.



One problem, many possible algorithms

Example problem

Given a collection of values, sort them in increasing order.

Possible algorithms

- Insertion sort
- Merge sort
- Quicksort
- Heap sort
- Counting sort

Question

If several algorithms can solve the same problem, how do we choose the most suitable one?



Choosing a suitable algorithm

Input characteristics

- Is the input small or large?
- Is the data almost sorted?
- Are the values integers?
- Is the value range small?
- Are there many repeated values?

Resource constraints

- How much memory is available?
- Is worst-case performance important?
- Is implementation simplicity important?
- Does the algorithm need to be stable?
- Does the data change over time?

Key idea

Algorithm choice depends on the input, the constraints, and the cost of time and memory.



Why do we study algorithms and complexity?

Algorithmic knowledge helps us answer

- Which algorithm is correct for this problem?
- Which algorithm is fast enough for this input size?
- Which algorithm uses acceptable memory?
- Which algorithm behaves well in difficult cases?

A good algorithm is not only correct; it must also be suitable for the data and the constraints of the problem.



Data Structures, Algorithms and Maths help

API Pricing

DeepSeek	ChatGPT
27x cheaper than ChatGPT! \$10 for the same workload.	\$270 for the same API services. A big difference!

linkup BUSINESS

Bigger datasets aren't always better

MIT researchers developed a way to identify the smallest dataset that guarantees optimal solutions to complex problems.

Adam Zewe | MIT News
November 18, 2025



<https://arxiv.org/pdf/2505.21692>



Course at a glance

General information

- **Title:** Data Structures and Algorithms in Python
- **Type:** Compulsory
- **Credits:** 3

Note:

- Slides will be uploaded to the LMS before the class
- There will be a tutorial every two lectures (scheduled by Instructors)
- Office hour (scheduled by Instructors)

Study hours

- Lectures: 45 hours
- Tutorials: 22.5 hours
- Self-study: 90 hours



Instructors and learning resources

Instructors

Assoc. Prof. Dr. Nguyen Trung Thanh
nguyenthanh@neu.edu.vn

Dr. Vu Duc Minh
minhvd@neu.edu.vn

M.Sc. Nguyen Thanh Hoang
hoangnt@neu.edu.vn

M.Sc. Dam Tien Thanh
thanhdtd@neu.edu.vn

M.Sc. Ta Dinh Quy
quytd@neu.edu.vn

Main textbooks

Goodrich et al. (2013), *Data Structures and Algorithms in Python*.

Additional resource

Cormen et al. (2022), *Introduction to Algorithms*.

Kleinberg and Tardos (2005), *Algorithm Design*.

Karumanchi (2015), *Data Structure and Algorithmic Thinking with Python*.



What you learn

Data structures

- ⑤ Arrays and Linked lists
- ⑥ Stacks and Queues
- ⑦ Hash tables and Heap
- ⑧ Trees and Graphs

Algorithms

- ① Sorting and Searching algorithms
- ② Traversal algorithms: DFS, BFS
- ③ Shortest path (dynamic programming)
- ④ Other algorithms (time permitting)

Applications will be discussed primarily in the context of machine learning and data science



Data structures - An illustration

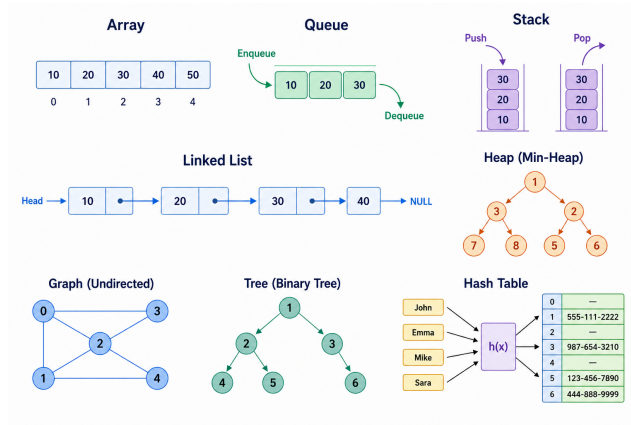


Figure 5: Example of data structures studied in the course



Grading methods

Assessment	Format/Evaluation	Weight
Attendance	Based on class participation	10%
Homework assignment	Written paper	20%
Midterm exam	01 Written exam	20%
Final exam	01 Written exam	50%

Note:

- 1 This is not a programming course.
- 2 There will likely be only a limited number of computer lab sessions.



Attendance and exam formats

Attendance

- At least 80% class attendance
- Active participation in class

Homework assignment

- Every week
- Handwritten on paper.
- Scan and upload to the LMS by the deadline.

Midterm Exam

- Written exam: 90 mins
- Week 8th (subject to change)

Final Exam

- Written exam: 90 mins
- Week 17th (subject to change)
- Condition: **Attendance_score** ≥ 5



How is DSA related to other courses?

Course	Main DSA Concepts
Algorithm Design	DP, graphs, complexity
Introduction to Database	B-trees, hashing, indexing
Advanced Database	Query optimization, graph processing
Introduction to AI	Trees, graphs, A*, BFS, DFS
Information Security	Hashing, string matching
Statistical Learning	KD-trees, heaps, optimization
Data Mining	Trees, graphs, hash tables
Big Data Technology	Distributed data structures, MapReduce
Software Engineering	ADTs, complexity analysis
Cloud Computing	Consistent hashing, distributed indexing



Key takeaway

If you skip DSA and go straight to AI frameworks, you may build applications faster—but without the fundamentals, you'll struggle to reason about performance, debug failures, and design scalable solutions. DSA gives you the foundation to understand, evaluate, and improve AI-generated code.



Any Question?

